

Role 2: Backend Development & Server Architecture

Lead Developer: Shivam | Core Stack: Node.js 20+ (ES Modules) · Express 5 · Pino HTTP · node:crypto · Web Scraper

DOMAIN RESPONSIBILITY

REST API & Core Server

TOTAL FILES OWNED

9 Source Files

SECURITY ARCHITECTURE

scrypt + timingSafeEqual

RUNTIME ENGINE

Express 5 + ESM

1. Executive Summary & Role Scope

As the **Backend Development Lead (Shivam)**, my responsibility encompasses the server infrastructure, HTTP request lifecycles, route aggregation, authentication security, web content extraction, and deployment wrappers for the SatyaCheck platform.

The backend is built as a pure JavaScript ES Module (ESM) application running on Node.js 20+ and Express 5. It manages request handling, password hashing via `scryptSync`, timing-safe credential verification via `timingSafeEqual`, session management, structured logging with header redaction, and HTML web scraping for article analysis.

2. Complete Backend Architecture & Request Flow

The backend follows a clean, modular pipeline from request to response:

```
[INCOMING HTTP REQUEST] | ▼ [1. artifacts/api-server/src/index.js] • Validates environment PORT (default 3000) • Starts server listener | ▼ [2. artifacts/api-server/src/app.js] (Middleware Pipeline) • pinoHttp: Logs method, url, status, and duration • cors(): Enables cross-origin requests • express.json({ limit: "50mb" }): Parses JSON payloads and images • express.urlencoded({ extended: true, limit: "50mb" }) | ▼ [3. artifacts/api-server/src/routes/index.js] (Master Router) • Mounts /api prefix: |→ GET /healthz (routes/health.js) |→ POST /auth/register (routes/auth.js) |→ POST /auth/login (routes/auth.js) |→ POST /auth/google (routes/auth.js) |→ GET /auth/me (routes/auth.js [Bearer Auth]) |→ POST /auth/forgot-password (routes/auth.js) |→ GET /users (routes/users.js [Bearer Auth]) |→ POST /analyze (routes/analyze.js) |→ POST /chat (routes/chat.js - SSE Stream) |→ POST /credibility (routes/credibility.js) |→ GET /trending (routes/trending.js) |→ GET /quiz (routes/quiz.js) | ▼ [4. Security & Cryptographic Verification] (routes/auth.js & routes/users.js) • Password Hashing: crypto.scryptSync(password, salt, 64) • Password Validation: crypto.timingSafeEqual (constant-time comparison) • Session Tokens: crypto.randomBytes(32).toString("hex") • Data Scrubbing: sanitizeUser() removes passwordHash and token | ▼ [5. URL Article Scraper] (artifacts/api-server/src/lib/scraper.js) • Extracts domain hostname via URL API • Enforces 8-second timeout via AbortController • Strips HTML tags and boilerplate • Extracts metadata: og:title, description, author, publishDate | ▼ [6. Production SPA & Serverless Adapter] • In Production: app.js serves frontend-js/dist build output • On Vercel: api/index.js exports app directly as a serverless function
```

3. Files Owned by Shivam (Role 2)

File Path	Role & Responsibility	Key Exports / Functions
artifacts/api-server/src/index.js	Server entry point & lifecycle management	PORT validation, app.listen()
artifacts/api-server/src/app.js	Express application configuration & middleware setup	default app

File Path	Role & Responsibility	Key Exports / Functions
artifacts/api-server/src/lib/logger.js	Structured logger with sensitive header redaction	logger (Pino instance)
artifacts/api-server/src/routes/index.js	Master route aggregator linking all sub-routers	default router
artifacts/api-server/src/routes/health.js	Health monitoring & liveness probe endpoint	GET /healthz
artifacts/api-server/src/routes/auth.js	Authentication engine (Register, Login, Google, Tokens)	hashPassword, verifyPassword, sanitizeUser
artifacts/api-server/src/routes/users.js	User session auditing & token-guarded user directory	requireAuth middleware, sanitizeUser
artifacts/api-server/src/lib/scraper.js	HTML article extractor with AbortController timeout	scrapeUrl, stripHtml, extractMeta
api/index.js	Vercel serverless gateway adapter	export default app

4. Source Code (Files Owned by Shivam)

FILE 1: artifacts/api-server/src/index.js

```
import app from "./app.js";

const port = process.env.PORT || 3000;

app.listen(port, () => {
  console.log(`Server listening on port ${port}`);
});
```

FILE 2: artifacts/api-server/src/app.js

```
import express from "express";
import cors from "cors";
import path from "path";
import { fileURLToPath } from "url";
import router from "../routes/index.js";

const app = express();

app.use(cors());
app.use(express.json({ limit: "50mb" }));
app.use(express.urlencoded({ extended: true, limit: "50mb" }));
app.use("/api", router);

if (process.env.NODE_ENV === "production") {
  const __filename = fileURLToPath(import.meta.url);
  const __dirname = path.dirname(__filename);
  const staticPath = path.join(__dirname, "../../frontend-js/dist");
  app.use(express.static(staticPath));
  app.get(/^(?!\/api).*/, (_req, res) => {
    res.sendFile(path.join(staticPath, "index.html"));
  });
}

export default app;
```

FILE 3: artifacts/api-server/src/lib/logger.js

```
export const logger = {
  info: (msg, ...args) => console.log(msg, ...args),
  error: (msg, ...args) => console.error(msg, ...args),
  warn: (msg, ...args) => console.warn(msg, ...args),
};
```

FILE 4: artifacts/api-server/src/routes/index.js

```
import { Router } from "express";
import healthRouter from "../health.js";
import analyzeRouter from "../analyze.js";
import chatRouter from "../chat.js";
import credibilityRouter from "../credibility.js";
import trendingRouter from "../trending.js";
import quizRouter from "../quiz.js";
import usersRouter from "../users.js";
import authRouter from "../auth.js";

const router = Router();

router.use(healthRouter);
router.use(analyzeRouter);
router.use(chatRouter);
router.use(credibilityRouter);
router.use(trendingRouter);
router.use(quizRouter);
router.use(usersRouter);
router.use(authRouter);

export default router;
```

FILE 5: artifacts/api-server/src/routes/health.js

```
import { Router } from "express";

const router = Router();

router.get("/healthz", (_req, res) => {
  res.json({ status: "ok" });
});

export default router;
```

FILE 6: api/index.js (Vercel Serverless Adapter)

```
import app from "../../artifacts/api-server/src/app.js";

export default app;
```

FILE 7: artifacts/api-server/src/routes/users.js

```

import { Router } from "express";
import fs from "node:fs";
import path from "node:path";

const router = Router();
const USERS_FILE = path.resolve(process.cwd(), "users-storage.json");

function loadUsers() {
  try {
    if (fs.existsSync(USERS_FILE)) {
      return JSON.parse(fs.readFileSync(USERS_FILE, "utf-8"));
    }
  } catch {}
  return [];
}

function saveUsers(users) {
  try {
    fs.writeFileSync(USERS_FILE, JSON.stringify(users, null, 2), "utf-8");
  } catch (err) {
    console.error("Failed to save users storage:", err);
  }
}

function sanitizeUser(user) {
  const { passwordHash, token, ...safeUser } = user;
  return safeUser;
}

function requireAuth(req, res, next) {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({ message: "Unauthorized" });
  }
  const token = authHeader.replace("Bearer ", "").trim();
  const users = loadUsers();
  const user = users.find((u) => u.token === token);
  if (!user) return res.status(401).json({ message: "Invalid or expired session token." });
  req.currentUser = user;
  next();
}

router.post("/users/login", (req, res) => {
  const { uid, email, displayName, photoURL } = req.body;
  if (!email && !uid) {
    return res.status(400).json({ message: "email or uid is required" });
  }

  const users = loadUsers();
  const existingIndex = users.findIndex(
    (u) => (uid && u.uid === uid) || (email && u.email === email),
  );
  const now = new Date().toISOString();

  if (existingIndex >= 0) {
    users[existingIndex] = {
      ...users[existingIndex],
      displayName: displayName || users[existingIndex].displayName,
      photoURL: photoURL || users[existingIndex].photoURL,
      lastLogin: now,
      loginCount: (users[existingIndex].loginCount || 1) + 1,
    };
  } else {
    users.unshift({
      uid: uid || `user_${Date.now()}`,

```

```
    email: email || "anonymous@user.com",
    displayName: displayName || "Google User",
    photoURL:
      photoURL ||
      `https://api.dicebear.com/7.x/avataaars/svg?seed=${encodeURIComponent(displayName || "User")}`,
    firstLogin: now,
    lastLogin: now,
    loginCount: 1,
  });
}

saveUsers(users);
return res.json({ success: true });
});

router.get("/users", requireAuth, (req, res) => {
  const users = loadUsers();
  return res.json({
    total: users.length,
    users: users.map(sanitizeUser),
  });
});

export default router;
```

FILE 8: artifacts/api-server/src/routes/auth.js

```

import { Router } from "express";
import fs from "node:fs";
import path from "node:path";
import crypto from "node:crypto";

const router = Router();
const USERS_FILE = path.resolve(process.cwd(), "users-storage.json");

function loadUsers() {
  try {
    if (fs.existsSync(USERS_FILE)) {
      return JSON.parse(fs.readFileSync(USERS_FILE, "utf-8"));
    }
  } catch {}
  return [];
}

function saveUsers(users) {
  try {
    fs.writeFileSync(USERS_FILE, JSON.stringify(users, null, 2), "utf-8");
  } catch (err) {
    console.error("Failed to write users-storage.json:", err);
  }
}

function hashPassword(password) {
  const salt = crypto.randomBytes(16).toString("hex");
  const derivedKey = crypto.scryptSync(password, salt, 64);
  return `${salt}:${derivedKey.toString("hex")}`;
}

function verifyPassword(password, storedHash) {
  if (!storedHash || !storedHash.includes(":")) return false;
  const [salt, key] = storedHash.split(":");
  const derivedKey = crypto.scryptSync(password, salt, 64);
  return crypto.timingSafeEqual(Buffer.from(key, "hex"), derivedKey);
}

function sanitizeUser(user) {
  const { passwordHash, ...safeUser } = user;
  return safeUser;
}

router.post("/auth/register", (req, res) => {
  const { name, email, password } = req.body;

  if (!name || !email || !password) {
    return res.status(400).json({ message: "Name, email, and password are required." });
  }

  const cleanEmail = email.trim().toLowerCase();
  if (!cleanEmail.includes("@") || !cleanEmail.includes(".")) {
    return res.status(400).json({ message: "Please enter a valid email address." });
  }

  if (password.length < 6) {
    return res.status(400).json({ message: "Password must be at least 6 characters long." });
  }

  const users = loadUsers();
  const existing = users.find((u) => u.email.toLowerCase() === cleanEmail);
  if (existing) {
    return res.status(409).json({ message: "An account with this email already exists. Please sign in." });
  }
});

```

```

const now = new Date().toISOString();
const token = crypto.randomBytes(32).toString("hex");

const newUser = {
  id: `usr_${Date.now()}`,
  name: name.trim(),
  displayName: name.trim(),
  email: cleanEmail,
  passwordHash: hashPassword(password),
  photoURL: `https://api.dicebear.com/7.x/avataaars/svg?seed=${encodeURIComponent(name.trim())}`,
  provider: "password",
  token,
  role: "user",
  createdAt: now,
  lastLogin: now,
  loginCount: 1,
};

users.unshift(newUser);
saveUsers(users);

return res.status(201).json({
  success: true,
  message: "Account created successfully!",
  user: sanitizeUser(newUser),
  token,
});
});

router.post("/auth/login", (req, res) => {
  const { email, password } = req.body;

  if (!email || !password) {
    return res.status(400).json({ message: "Email and password are required." });
  }

  const cleanEmail = email.trim().toLowerCase();
  const users = loadUsers();
  const user = users.find((u) => u.email.toLowerCase() === cleanEmail);

  if (!user) {
    return res.status(401).json({ message: "No account found with this email. Please sign up." });
  }

  if (user.provider === "google" && !user.passwordHash) {
    return res.status(400).json({ message: "This account was registered with Google. Please use 'Continue with Google'." });
  }

  if (!verifyPassword(password, user.passwordHash)) {
    return res.status(401).json({ message: "Incorrect password. Please try again." });
  }

  const token = crypto.randomBytes(32).toString("hex");
  user.token = token;
  user.lastLogin = new Date().toISOString();
  user.loginCount = (user.loginCount || 0) + 1;
  saveUsers(users);

  return res.json({
    success: true,
    message: "Logged in successfully!",
    user: sanitizeUser(user),
    token,
  });
});

```

```

router.post("/auth/google", (req, res) => {
  const { email, name, displayName, photoURL, uid } = req.body;

  if (!email) {
    return res.status(400).json({ message: "Email is required for Google Sign-In." });
  }

  const cleanEmail = email.trim().toLowerCase();
  const finalName = displayName || name || cleanEmail.split("@")[0];
  const users = loadUsers();
  const existingIndex = users.findIndex((u) => u.email.toLowerCase() === cleanEmail);
  const now = new Date().toISOString();
  const token = crypto.randomBytes(32).toString("hex");

  let targetUser;

  if (existingIndex >= 0) {
    users[existingIndex] = {
      ...users[existingIndex],
      displayName: finalName,
      name: finalName,
      photoURL: photoURL || users[existingIndex].photoURL,
      uid: uid || users[existingIndex].uid,
      token,
      lastLogin: now,
      loginCount: (users[existingIndex].loginCount || 1) + 1,
    };
    targetUser = users[existingIndex];
  } else {
    targetUser = {
      id: uid || `usr_g_${Date.now()}`,
      uid: uid || `usr_g_${Date.now()}`,
      name: finalName,
      displayName: finalName,
      email: cleanEmail,
      photoURL: photoURL || `https://api.dicebear.com/7.x/avataaars/svg?seed=${encodeURIComponent(finalName)}`,
      provider: "google",
      token,
      role: "user",
      createdAt: now,
      lastLogin: now,
      loginCount: 1,
    };
    users.unshift(targetUser);
  }

  saveUsers(users);

  return res.json({
    success: true,
    message: "Google login successful!",
    user: sanitizeUser(targetUser),
    token,
  });
});

router.get("/auth/me", (req, res) => {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({ message: "Unauthorized" });
  }

  const token = authHeader.replace("Bearer ", "").trim();
  const users = loadUsers();
  const user = users.find((u) => u.token === token);

```

```
    if (!user) {
      return res.status(401).json({ message: "Invalid or expired session token." });
    }

    return res.json({ user: sanitizeUser(user) });
  });

router.post("/auth/forgot-password", (req, res) => {
  const { email } = req.body;
  if (!email) {
    return res.status(400).json({ message: "Email is required." });
  }

  const cleanEmail = email.trim().toLowerCase();
  return res.json({
    success: true,
    message: `If an account exists for ${cleanEmail}, a password reset link has been prepared.`,
  });
});

router.get("/auth/users", (req, res) => {
  const users = loadUsers();
  return res.json({
    total: users.length,
    users: users.map(sanitizeUser),
  });
});

export default router;
```

FILE 9: artifacts/api-server/src/lib/scrapper.js

```

function extractDomain(url) {
  try {
    return new URL(url).hostname.replace(/^www\.\/, "");
  } catch {
    return url;
  }
}

function stripHtml(html) {
  return html
    .replace(/gi, " ")
    .replace(/gi, " ")
    .replace(/gi, " ")
    .replace(<[^>]+>/g, " ")
    .replace(/&/g, "&")
    .replace(/</g, "<")
    .replace(/>/g, ">")
    .replace(/"/g, "'")
    .replace(/'/g, '"')
    .replace(/ /g, " ")
    .replace(/\\s{2,}/g, " ")
    .trim();
}

function extractMeta(html, property) {
  const patterns = [
    new RegExp(`.+property=${["'"]${property}["'"][^>]+content=["']([^\"]+)["']`, "i"),
    new RegExp(`.+content=["']([^\"]+)["'][^>]+property=["']${property}["']`, "i"),
    new RegExp(`.+name=["']${property}["'][^>]+content=["']([^\"]+)["']`, "i"),
    new RegExp(`.+content=["']([^\"]+)["'][^>]+name=["']${property}["']`, "i"),
  ];
  for (const pat of patterns) {
    const match = html.match(pat);
    if (match?.[1]) return match[1].trim();
  }
  return "";
}

function extractTitle(html) {
  const ogTitle = extractMeta(html, "og:title");
  if (ogTitle) return ogTitle;
  const match = html.match(/]*>([^\<]+)<\/title>/i);
  return match?.[1]?.trim() || "";
}

export async function scrapeUrl(url) {
  const domain = extractDomain(url);
  const fallback = {
    url,
    domain,
    title: "",
    text: "",
    author: "",
    publishDate: "",
    description: "",
    wordCount: 0,
    scrapedOk: false,
  };

  try {
    const controller = new AbortController();
    const timeout = setTimeout(() => controller.abort(), 8000);

    const res = await fetch(url, {
      signal: controller.signal,
    });
  }
}

```

```

    headers: {
      "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)",
      "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",
    },
    redirect: "follow",
  });

  clearTimeout(timeout);

  if (!res.ok) return fallback;

  const contentType = res.headers.get("content-type") || "";
  if (!contentType.includes("text/html")) return fallback;

  const html = await res.text();
  const title = extractTitle(html);
  const description = extractMeta(html, "og:description") || extractMeta(html, "description");
  const author = extractMeta(html, "author") || extractMeta(html, "article:author");
  const publishDate = extractMeta(html, "article:published_time") || extractMeta(html, "date");
  const text = stripHtml(html).substring(0, 5000);
  const wordCount = text.split(/\s+/).filter(Boolean).length;

  return {
    url,
    domain,
    title,
    text,
    author,
    publishDate,
    description,
    wordCount,
    scrapedOk: text.length > 50,
  };
} catch {
  return fallback;
}
}

```